



Mini-Assignment 2

Advanced Techniques for Language Models

Cross-Judge RLAIIF and DPO Alignment of a Domain-Adapted Small Language Model

Index

1	Introduction	1
2	Pipeline Overview	2
2.1	A1: domain adaptation	2
2.2	A2: alignment	3
2.3	A3: the system, and the model-choice justification	3
3	System Design	4
3.1	Architecture Overview	4
3.2	Component 1: Hybrid RAG over a Domain Knowledge Base	5
3.3	Component 2: Multi-Step Agentic Orchestration	6
3.4	Component 3: LLM-as-Judge	7
3.5	Component 4: Performance Optimisation	7
3.6	Candidate Matching (Designed Extension)	8
3.7	Chain-of-Thought (Supporting Component)	9
3.8	Models	9
4	Implementation Details	9
4.1	Backend	9
4.2	Frontend and UX	10
4.3	Projects	11
4.4	Infrastructure and Live Monitoring	12
5	Evaluation	12
5.1	What “the system works” means	12
5.2	Baselines and results	13
5.3	Qualitative analysis	14
6	Critical Discussion	14
6.1	What worked	15
6.2	Honest limitations	15
6.3	Trade-offs	16
6.4	What we would do differently, and next steps	16
7	Reflection on the Three-Stage Arc	16
8	Conclusion	17

1 Introduction

This report documents the final stage of a project whose goal is to turn a hiring need, expressed in natural language, into a coherent pack of HR documents. Mini-Assignment 1 adapted a small, open-source language model to the domain of IT job postings. Mini-Assignment 2 taught that checkpoint to follow recruiter instructions through supervised fine-tuning and DPO-based preference alignment. The present work, *job2cool*, integrates the resulting aligned model into a complete application. Its user-facing persona is Diana, an HR assistant that interacts with the recruiter to produce a Job Offer, a set of Technical Interviews, an Onboarding Plan, and a Cultural and Team Fit assessment, all grounded in a company knowledge base, written live into editable documents, and cited with click-through links to the exact source PDF page and bounding box.

At the core of the system sits `ma2-360m-dpo-b01`, the SmolLM2-360M checkpoint adapted in Mini-Assignment 1 and DPO-aligned in Mini-Assignment 2. Around it, *job2cool* adds a general-purpose orchestrator, `gemma-4` (E4B-class, 128 K context), and four additional course components beyond the alignment work already done, two more than the assignment requires:

1. **Retrieval-augmented generation:** concurrent vector and graph search over a knowledge base of ten role-specific onboarding domains and a 210,048-CV candidate corpus, grounding output in company material and enabling clickable citations to source PDFs with page-level bounding-box overlays.
2. **Tool use and agentic orchestration:** `gemma-4` LLM-decides each pipeline step (from conversational memory resolution, through section classification, to per-section retrieval and composition) with no rule- or regex-based routing.
3. **LLM-as-a-judge:** every response is scored for faithfulness and answer-relevance against the retrieved evidence, surfaced per turn in the UI.
4. **Performance optimisation:** llama.cpp quantised inference under a model router, plus a root-caused configuration fix for a multi-slot KV-buffer memory failure that had crashed the host four times during development.

A fifth course technique, advanced reasoning, is present but counted as supporting rather than headline: `gemma-4` emits inline chain-of-thought that the orchestrator both consumes (for its routing decisions) and surfaces live in a Thinking panel (Section 3.7). All models in the request path are open-source and quantised; no closed API is used anywhere in the serving path.

The system is functional end-to-end, with two clear entry points: a recruiter-facing web UI and a command-line candidate-ingestion tool. In the UI, Diana identifies the role from conversational context, resolves the matching knowledge-base domain, determines which of the four documents were requested, retrieves evidence concurrently from vector and graph indices, streams a reasoning-aware introduction, and generates each document progressively in a tabbed

workspace whose citations open the source PDF at the cited page with a bounding-box highlight. The interface is a left-nav-rail application (Workspace, Projects, Candidates, Agents, Skills, Tools, and Knowledge Base) served over fifteen request-path containers across three Docker networks, authenticated via Google identity through an `oauth2-proxy` gateway. A Candidates browser gives direct access to the indexed CV corpus; a Projects view provides private and shared project management with full-fidelity replay of every past turn, including its reasoning and grounding panels; and a live infrastructure health map monitors all services event-driven.

Although development and evaluation focused on IT recruitment, the architecture does not depend on characteristics specific to this sector. The domain-organised knowledge base, the domain-resolution mechanism, and the orchestration pipeline are all independent of the concrete content of those domains, so the same system could in principle be adapted to other recruitment areas by replacing the knowledge base and the specialised drafting model.

2 Pipeline Overview

The project is a single pipeline organised into three stages, from a generic pretrained model to the system presented here. Each stage adds a capability to the previous checkpoint without discarding what was already acquired (Figure 1).

STAGE A1 domain adaptation

SmolLM2-360M continued-pretrained on ~12k Djinni IT postings (LoRA)
 in-domain PPL 16.37 to 11.38 (-31%); out-of-domain unchanged
 yields a domain-adapted 360M checkpoint (merged LoRA)

STAGE A2 alignment

SFT on ~7.5k request-posting pairs (validation PPL 11.66 to 4.54)
 DPO via RLAIIF, beta = 0.10; win-rate 8-0 vs SFT; DPO val PPL 6.01
 yields ma2-360m-dpo-b01, the Job-Offer specialist (~430 tok/s)

STAGE A3 the system (this work)

gemma-4 E4B orchestrator + ma2-360m-dpo-b01 (live Job Offer drafter)
 Component 1 Hybrid RAG: vector + graph, 10 domains + 210,048 CVs
 Component 2 Agentic pipeline: 8 LLM-decided steps, memory, guard
 Component 3 LLM-as-judge: RAGAS faithfulness + answer-relevance
 Component 4 Performance optimisation: llama.cpp, router, slot cap
 Supporting Chain-of-thought reasoning, in the Thinking panel

Figure 1: The three-stage arc from raw pretrained model to job2cool. The A3 stage is not a black box: it expands into the live, health-monitored dependency graph the system renders of itself (Section 4.4).

2.1 A1: domain adaptation

Mini-Assignment 1 continued the pretraining of SmolLM2-360M on roughly 12,000 IT job postings from the Djinni dataset (141,897 postings in the full release), shifting the model’s

language distribution toward the vocabulary and conventions of this type of posting. Under identical conditions, LoRA beat full fine-tuning on in-domain perplexity (11.38 versus 13.12, against a base of 16.37) while training only 2.3% of parameters in roughly a third of the time. Out-of-domain perplexity, measured on LinkedIn postings, remained practically unchanged for both, confirming the adaptation was IT-specific rather than a general drift. The merged LoRA checkpoint became the A2 starting point.

2.2 A2: alignment

Mini-Assignment 2 turned that domain-fluent but instruction-blind checkpoint into a recruiter-instructable drafter. Supervised fine-tuning on roughly 7,500 request-and-posting pairs drove validation perplexity from 11.66 to 4.54 and made the model reliably emit the four Job-Offer sections (Summary, Required Skills, Responsibilities, and Requirements) in Markdown. DPO via Reinforcement Learning from AI Feedback then ranked sampled candidates against a rubric covering faithfulness, structural completeness, language quality, and absence of repetition; a sweep over four values of β peaked at $\beta = 0.10$, yielding **ma2-360m-dpo-b01**, which won 8 of 20 held-out comparisons against the SFT model (0 losses) under a Granite judge with order-swap. DPO validation perplexity settled at 6.01, higher than SFT's 4.54, as expected under KL regularisation.

Two A2 findings carry directly into A3. First, the aligned model's behaviour depends strongly on the prompt template and on inference parameters, most notably the repetition penalty, whose absence caused repetition collapses, so A3 must keep the same inference rigour when calling the model through a new orchestration layer. Second, judge-based evaluation has a discrimination ceiling: a smaller judge distinguished broad comparisons (base versus aligned) well but was unreliable on close comparisons between similar checkpoints, which motivates an evaluation design that does not rest on a single judge (Section 5).

2.3 A3: the system, and the model-choice justification

Mini-Assignment 3 uses **ma2-360m-dpo-b01** exactly as delivered, without further modification, assigning it one specific role: drafting Job Offers. Everything else, namely interpreting the conversation, resolving the knowledge-base domain, deciding which documents to produce, retrieving evidence, composing each section (including refining the A2 draft), and attaching citations, is handled by the orchestration layer and the **gemma-4** general model.

The assignment permits switching to a different open-source model or running a further adaptation pass. **job2cool** does neither as a replacement: it retains the A2 model and adds **gemma-4** alongside it, and the justification is capability-versus-task fit. The A2 model is a 360M-parameter specialist, excellent and fast at the one task it was aligned for (natural-language need into a Markdown Job Offer), but with no tool use, no long-context planning, and no general composition ability for onboarding, interview, or culture content; asking a 360M model to drive

the whole pipeline would fail. **gemma-4** is the open-source orchestrator (4B-class, 128 K context) that runs the agentic plan, composes the RAG-grounded sections, and refines the A2 draft. Both are co-resident on **agent_server** and selectable per request by model id, so the specialist and the orchestrator coexist rather than one superseding the other. This is the honest reading of the arc: the final stage is an orchestration-and-grounding layer that makes the aligned model *usable*, not a larger model that makes it redundant. Whether **gemma-4**'s composition step masks the A2 alignment in the final output is one of the questions the evaluation is built to answer (Section 5, Section 7).

Retrieval and orchestration therefore operate on a different axis from the earlier work, which improved the quality of one output (the Job Offer) in isolation. They address what an isolated aligned model cannot resolve regardless of its quality: knowledge of the company's actual material, determining what a request is asking for, holding context across a conversation, and proving through citation that a claim has an identifiable source.

3 System Design

The assignment requires at least two course techniques layered on top of the A1 fine-tuning and A2 alignment (which do not themselves count). **job2cool** integrates four, each mapped to a named course bucket and each solving a problem the aligned model cannot solve on its own: retrieval-augmented generation (Section 3.2), tool use and agentic orchestration (Section 3.3), LLM-as-a-judge (Section 3.4), and performance optimisation (Section 3.5). A candidate-matching layer (Section 3.6) deepens the first two, and chain-of-thought reasoning (Section 3.7) is present as a supporting fifth. The components are not independent add-ons. The agentic layer decides *when* to call the A2 drafter and *what* to retrieve; RAG supplies the evidence and the citations the composition is grounded in; the judge measures whether that grounding held; and performance optimisation is what lets the orchestrator, specialist, embedder, and reranker run together on one GPU and ingest the corpus the retrieval depends on.

3.1 Architecture Overview

job2cool follows an α -adapter pattern: **job2cool-backend** (FastAPI) preserves the exact server-sent-events chat contract that the Diana widget expects, while orchestrating across shared services it neither owns nor modifies. The backend owns a fixed set of API prefixes and reverse-proxies every other call to **kb-service**, a knowledge-base gateway that composes the management and document APIs of the underlying engines (**noted-rag**, **noted-graph**, and **noted-arcadedb**) and falls through to **noted** only for source-PDF file serving. Model inference runs on **agent_server** (llama.cpp), and tools and skills are hosted by **mcp-service**. An nginx reverse proxy (**proxy_server**) behind **oauth2-proxy** gates all external traffic and provides Google identity through **/api/job2cool/me**. Voice services (STT, TTS, and avatar) route through nginx exclusively, never through the backend, which is why audio works on the deployed

origin but not on a bare `localhost:4920`.

The backend joins two external Docker networks (`noted-network` and `logus2k_network`); a third, `mcp_internal`, isolates the headless-browser web-search backend. The full request-path topology, fifteen containers rendered with the browser as a sixteen-box map, is live-health-monitored by the system itself (Section 4.4), so the same artifact is both the design diagram and a running dashboard.

The guiding constraint was **copy-only**: `job2cool` builds from copies of the `cv` and `noted` client files and consumes the shared services in place without modifying them. The single authorised exception is a one-line **overflow**: `hidden` fix in `noted`'s `panels.css`. This is a real engineering constraint with real costs, since it forced new capabilities in the shared engines to be *additive* (new endpoints rather than edits, Section 4.1), and it is revisited in the trade-offs (Section 6.3). The decision to reuse the serving stack rather than rebuild chat, tool-calling, retrieval, and live-document buffers is what keeps the arc honest: A3 is an integration layer around the aligned model, not a from-scratch inference stack.

3.2 Component 1: Hybrid RAG over a Domain Knowledge Base

The knowledge base is a set of per-domain corpora, each a ChromaDB collection (`<domain>_corpus`) plus a graph project. Ten of eleven role-onboarding families are populated today: `architect`, `backend`, `data`, `devops`, `frontend`, `general`, `ml_ai`, `mobile`, `qa`, and `security`; only `embedded` is not yet created. A separate vector-only collection, `jobs_candidates__corpus`, holds the 210,048-CV candidate corpus, and several thematic corpora (`jobs_eng_culture`, `ai_and_jobs`, `prod_mng`, `sw_arch`, and research collections) are also indexed.

Retrieval for each turn is **concurrent**. `graph_and_vector_search` embeds the query once and runs vector and graph retrieval in parallel via `asyncio`, then merges them into a single evidence block. Vector search is dense retrieval over the resolved domain's collection (`bge-m3` cosine HNSW) followed by `bge-reranker-v2-m3` cross-encoder re-ranking; its query is LLM-rewritten first (reusing the existing `cv_query_rewriter` preset) because a cleaned, keyword-focused query retrieves better than a conversational sentence. Graph search over the same domain uses the *raw* question, since entity-name search handles conversational phrasing better than a rewritten phrase, and returns entities, relationships, and chunk excerpts carrying page number and bounding-box regions. Citation tags distinguish the sources: `[markdown_chunk:hex]` for chunks (dense and graph-grounded), `[E:id]` for entities, and `[R:src>type>tgt]` for edges. As an honest design note, the merged set is *not* cross-reranked after the join: vector and graph results are concatenated, not globally re-scored, and a unified post-merge rerank is recorded as future work.

The citation system closes the provenance loop. Clicking a `[markdown_chunk:hex]` badge calls `/api/citation/{tag}`, which resolves through `noted-graph` `/chunk/{tag}` to a source path, a page number, and bbox regions; the frontend (`js2c/pdfcite.js`) opens a PDF split pane, renders the cited page with `pdf.js`, and paints the overlay via `viewport.convertToViewportRectangle`

(handling the PDF bottom-left to top-left flip), re-rendering on resize. Bbox highlighting is available only for graph-grounded excerpts; dense-search-only chunks open the PDF at the inferred page without a box, a property of the retrieval source rather than a rendering bug.

RAG was chosen over further fine-tuning because the onboarding, interview, and culture knowledge is broad and still growing: retrieval lets it change without retraining, and the graph half yields per-chunk citations to a PDF page and bounding box that fine-tuning fundamentally cannot provide. This addresses *grounding and provenance*, an axis orthogonal to the *writing quality* A1/A2 addressed (the basis of the Section 7 redundancy argument).

3.3 Component 2: Multi-Step Agentic Orchestration

`orchestrator.run_chat` is a fully LLM-decided pipeline of eight steps, where every routing decision is a model call with a tight contract, not a rule or a regex:

1. `_resolve_need` rewrites the request into a self-contained sentence using the last six turns, folding role and skills mentioned earlier into the current ask. This is the memory component an isolated aligned model lacks.
2. `_role_label` extracts the job title (`gemma-4`, `think=False`). If no concrete role is found, Diana asks which role and returns early, generating nothing, a guard against fabricating a package for an unspecified role.
3. `resolve_onboard_domain` classifies the role into one of eleven `ONBOARD_FAMILIES`, then confirms via `available_corpus_domains` (a lazy, process-lifetime cache of the live `*__corpus` collections) that the matching `jobs_onboard_<family>` actually exists, with fallback to `jobs_onboard_general`, then any existing `jobs_onboard_*`, then a default. Routing is grounded in what the KB really contains.
4. `_requested_sections` classifies which of {offer, interview, onboarding, culture} the request asks for. This replaced an earlier keyword heuristic that misfired on substrings (“full-stack” falsely matching a “full package” trigger), a concrete case of moving a brittle rule to an LLM decision.
5. **Streamed intro** confirms the role and deliverables via `llm_stream`. The raw stream, including reasoning, is forwarded to the Thinking panel, and a `<voice>` tag is appended so the avatar speaks a brief summary.
6. **Per-section loop.** For each deliverable, a buffer tab is created lazily (tabs appear one by one as sections complete); concurrent hybrid RAG retrieves over the resolved domain; for the Job Offer, if `ma2` is enabled, the A2 specialist drafts via `llm_complete` (`DPO_MAX` about 1200), and if both MA2 and Gemma are selected the raw A2 draft is written to a separate “Job Offer (MA2)” buffer for side-by-side comparison; `gemma-4` then composes the section

(`SECTION_MAX=8192`), refining the A2 draft when one exists; and `_cited_sources` appends a clickable Sources line.

7. `_closing_note` emits a deterministic summary of what landed and which domain was used, one grounded LLM sentence flagging thin KB coverage, and a nudge toward the not-yet-requested deliverables, making gaps visible rather than hiding them.
8. `cache.put_turn` records the full turn for the judge and graph-trace endpoints without re-querying the engines.

A fixed pipeline would force all four documents on every request and could not honour “just the job offer” or context-dependent follow-ups (“now make the onboarding remote friendly”). The LLM-decided design buys that flexibility at the cost of determinism (harder to unit-test), a trade-off recorded in Section 6.3.

3.4 Component 3: LLM-as-Judge

Every turn is scored by a RAGAS-style judge (`/api/score_answer`) that rates the composed documents against the retrieved evidence on faithfulness and answer-relevance, with a rationale, surfaced in a Score panel. For a grounded generator the load-bearing property is faithfulness-to-source, and an LLM judge gives a scalable signal of it that a deterministic “did the citation resolve?” check does not. The default `cv_rag_judge` preset was tried first but took roughly 400 seconds per call on this stack; the replacement, `gemma-4` with an explicit JSON instruction and `think=False`, runs in under ten seconds with consistent structured output. The acknowledged limitation, doubly relevant given the A2 judge-discrimination ceiling, is that `gemma-4` judges its own output; there is no cross-judge isolation.

3.5 Component 4: Performance Optimisation

All models run on `llama.cpp` (the `llama-vision` router) with quantised GGUF weights, namely `gemma-4` at Q4_K_XL, the A2 model, `bge-m3` at Q8, and `bge-reranker` at Q8, co-resident on one 24 GB-class GPU under a model router with a bounded RAM cache and `--models-max 4`. Quantisation plus a shared router is what lets a 4B orchestrator, a 360M specialist, an embedder, and a reranker fit on a single GPU at all. The headline work was making this stack survive a 210,048-CV bulk-embedding job, which OOM-killed the host four times.

The embedder’s RSS climbed without bound, observed at 80–84 GB for a roughly 2 GB model, and the path to the fix is reported honestly because it included two wrong diagnoses. The first, an “oversized embed batch” theory, led to token-budgeted sub-batching that appeared to work on a 13,000-embed test but still failed at 45,000. The second, “Cyrillic token explosion,” was disproven directly: the corpus is about 0.1% Cyrillic and the worst-case batch reached only about 4,900 tokens. Controlled experiments found the real cause: serial ingest stayed flat at about 3.7 GB, but the moment concurrent `/search` traffic overlapped the ingest, RSS ratcheted up in steps

(3.8, 8.7, 13.8, 24, then 44 GB) and never released. The server logs showed the mechanism: `total_slots = 4` with `n_ctx_per_seq = 8192`. bge-m3 ran with four slots, each allocating and *retaining* a compute/KV buffer sized for the 8192-token context; every overlapping request (a search, a chat turn, a stray health probe) activated another slot and grabbed another multi-GB buffer. It was never the data; it was concurrency multiplied by a retained per-slot buffer.

The fix is one correctness-preserving line in `agent_server/data/agent_config.json`:

```
{ "parallel": 1 }
```

Reducing the context would have shrunk each buffer but truncated any caller sending a long sequence, a correctness regression on a *shared* embedder. Capping slots to one instead leaves the 8192 context untouched: one slot means one buffer means bounded memory, and concurrent embeds queue rather than each allocating, at the cost of embedding parallelism (latency under simultaneous load) only.

Table 1: Embedder memory before and after the slot-cap fix.

	Before (4 slots)	After (1 slot)
bge-m3 RSS under load	80 GB, host OOM (4x)	2.2 GB, flat
Per-sequence capacity	8192 tokens	8192 tokens (unchanged)
Cost	n/a	concurrent embeds serialise

The decisive stress test, 13,208 embeds plus 6 concurrent search threads (the exact overlap that hit 80 GB), now peaks at 2,238 MB and stays flat, and the full ingest completed with read-back acknowledgement equal to the indexed count. Without this fix the candidate corpus could not have been ingested at all.

3.6 Candidate Matching (Designed Extension)

A second HR capability extends the system from a hiring-package generator toward a recruiter-workflow assistant: once Diana has drafted a Job Offer, the natural next question is who the company already has who fits the role. As designed, a `match_candidates` tool takes a role spec (family, seniority, skills, minimum English level) plus `top_k` and a `score_threshold`, and runs a three-stage hybrid retrieval. A ChromaDB structured pre-filter on `primary_keyword`, `english_level`, and `experience_years` narrows the 210k pool exactly and cheaply; vector search ranks the filtered subset by cosine similarity; and `bge-reranker-v2-m3` produces the final match score. The orchestrator then *branches on the result*: with a strong internal candidate the downstream sections are composed candidate-aware (Interview targets missing skills, Onboarding calibrates to seniority), otherwise they stay role-generic and a Candidate Matches section recommends external sourcing. That state-dependent branch is the deepest form of the agentic and reasoning components.

As built, the retrieval substrate is complete and verified: 210,048 CVs vector-indexed with structured metadata, a working semantic search, and a human-facing Candidates browser. But

the `match_candidates` tool and the conditional composition are *deferred* (Section 4.1, Section 6.2). This is the single largest gap between as-designed and as-built, stated plainly.

3.7 Chain-of-Thought (Supporting Component)

`gemma-4` natively emits inline `<think>` reasoning. The orchestrator relies on it for the conditional and section-selection steps and surfaces it live in the Thinking panel during the streamed intro. It is counted as supporting rather than headline because it is the model’s native behaviour rather than an engineered strategy such as self-consistency or tree-of-thoughts. Crucially, it is also a load-bearing *constraint*: on every non-streaming utility call the orchestrator sets `think=False` (via `chat_template_kwargs: {enable_thinking: False}`), because `gemma-4` otherwise spends its entire token budget inside the reasoning block and returns empty visible content. Role and section classification, query rewriting, domain resolution, the closing note, and the judge therefore cannot use chain-of-thought, a concrete capability-versus-stability trade-off (Section 6.3).

3.8 Models

Table 2: Models in the request path. All open-source and quantised; no closed API is used.

Model	Role	Format	Context
<code>gemma-4</code> (E4B)	Orchestrator, composer, judge, rewriter	Q4_K_XL GGUF	128K
<code>ma2-360m-dpo-b01</code>	Job Offer drafter (A1+A2 checkpoint)	Q8 GGUF	n/a
<code>bge-m3</code>	Dense embeddings (1024-dim, cosine)	Q8 GGUF	8192
<code>bge-reranker-v2-m3</code>	Cross-encoder re-ranking	Q8 GGUF	n/a

The orchestrator runs with `think=True` for generation and `think=False` for every utility call (Section 3.7). `ma2-360m-dpo-b01` drafts at roughly 430 tokens per second (about 0.7s per response); its `repetition_penalty=1.3` is preserved in the serving configuration, without which the model collapses into repetition loops, as documented in Mini-Assignment 2 and reproduced during integration testing.

4 Implementation Details

4.1 Backend

`job2cool-backend` is a FastAPI application that owns a precise set of `/api` prefixes (`health`, `chat` for SSE orchestration, `citation`, `graph_trace`, `score_answer`, `buffers`, `job2cool` for identity, Projects, and Candidates, `agents`, and `mcp`) and reverse-proxies everything else to `kb-service`, with `noted` fall-through for PDF files. Seven modules implement it: `main.py` (app, routes, proxy, Socket.IO mount), `orchestrator.py` (the agentic pipeline), `services.py` (async LLM and hybrid-RAG clients), `buffers.py` (live documents), `cache.py` (chunk and turn caches), `health_monitor.py` (the probe loop), and `socketio_relay.py` (KB-progress and health pushes).

Live documents are `DocBuffer` objects, in-memory and one per deliverable, with an `asyncio` pub/sub that broadcasts each change over SSE (`/api/buffers/events/stream`), so the workspace re-renders the document live, tab by tab. Token caps are conservative output bounds, not context limits: `SECTION_MAX=8192` (section composition), `INTRO_MAX=4096` (streamed intro), `DPO_MAX` about 1200 (A2 draft), and `SUMMARY_MAX` about 320 (closing note); `gemma-4`'s context window is 131,072 tokens, so none of these are system-level constraints. Two FIFO caches in `cache.py` keep the judge and graph endpoints cheap: a chunk cache (max 2,048, keyed by `sha1(chunk_id)[:12]`) so citation resolution does not re-query, and a turn cache (max 256, keyed by `turn_id`) holding the full turn so `/api/score_answer` and `/api/graph_trace` operate on a finished turn without re-running retrieval. One honest limitation lives here: `/api/buffers/id/save` acknowledges but does not persist to disk or the KB (a stub), though work still round-trips through the Projects store (Section 4.3).

Candidate ingestion. `scripts/ingest_candidates.py` streams the `lang-uk/recruitment-dataset-candidate-profiles-english` Parquet file (210,250 rows), drops 202 sub-50-character junk profiles, and stores the remaining 210,048 as *single-chunk* documents. The design originally assumed section-chunked CVs, but the real CVs are tiny (median about 110 words, or 750 characters), so one vector per whole CV is both correct and cheaper; the atomic retrieval unit is “this candidate,” and a whole CV fits easily in `bge-m3`'s 8192-token window. The corpus is also *vector-only*: full graph entity-extraction over 210k CVs benchmarked at about 2.3 days of GPU time and the matcher never reads the graph, so it was deliberately skipped. Structured fields (`primary_keyword`, `english_level`, `experience_years`, and `position`) are stored as ChromaDB metadata, the substrate for the Section 3.6 pre-filter, and deterministic IDs (`cand-{uuid}`) make re-runs idempotent.

Honouring the copy-only guardrail, the bulk ingest required two *additive* endpoints on `otedrag` rather than edits: `POST /upsert_records` (bulk embed and upsert with the caller's own flat metadata, token-budgeted sub-batching, `skip_existing`, and a **read-back verification** that re-reads the written IDs from ChromaDB before acknowledging, so a 200 means rows confirmed present) and `POST /list_records` (paginated browse, powering the Candidates view). The final segment reported `indexed 43008 (read-back ack 43008)`, every written row DB-confirmed.

The candidate corpus shares a lineage with the A1/A2 training data: both come from the Djinni platform (job postings for A1/A2, candidate profiles for A3) and share an identical structured vocabulary (Primary Keyword, English Level, and Experience Years), so a posting's requirements filter directly into a candidate query with no schema translation.

4.2 Frontend and UX

The frontend is a custom single-page application (`frontend/index.html`) baked into the Docker image at build time, so a frontend or backend change redeploys with a single `docker compose up -d --build job2cool-backend`. Layout is a full-viewport flex row: a left nav rail

and a main area with a workspace top bar, a tab bar, and a split document/PDF pane. Gold (`--primary: #ffe19b`) is the accent; the TalentForge AI mockup is the *layout* reference, not the colour spec. Each nav view is a small module under `js2c/`, and Diana is a patched copy of the `cv-chat.js` widget.

The left rail, verified against the running build, is **Workspace, Projects, Candidates, Agents, Skills, Tools, Knowledge Base**, and **Company Profile** (the one remaining placeholder), with Help & Support, Settings, and the authenticated-user profile at the bottom. Diana is patched in five places to live here: the chat POST carries `JOB2COOL_CONFIG`; a new-turn signal resets tabs; a citation with a source path opens the PDF split pane (`JOB2COOL_OPEN_PDF`) instead of an in-page highlight; asset paths are base-relative (`document.baseURI`) for `/job2cool/` sub-path deployment; and the persona becomes Diana with its own greetings. The widget retains the full stack: the Thinking, Graph, and Score panels, citations, and STT/TTS/avatar.

The Workspace shows one tab per generated deliverable (created lazily as each section completes) and an “All Documents” overview presenting the deliverables as a cards grid with a Generation Progress checklist that updates live, the mockup’s stepper wired to the real pipeline. The Job Offer tab shows a Gemma/MA2 segmented toggle when both versions exist. Per-tab actions are Copy Markdown and PDF export.

The remaining nav views are each a `js2c` module: `kb.js` (Knowledge Base domain manager with live Socket.IO build progress), `candidates.js` (the Candidates browser), `agents.js` (CRUD over the `job2cool_orchestrator`, `_composer`, and `_judge` presets that drive Diana), `mcp.js` (the Skills and Tools admin served through `mcp-service`), `help.js` and `architecture.js` (the live health map), `pdfcite.js` (the PDF and bbox renderer), and `sidepanel.js`.

Candidates browser. `candidates.js` renders a paginated browse over all 210k CVs, an Enter-to-search semantic retrieval whose results carry a match-percentage chip (the reranker score), and a per-CV detail side-panel. One bug surfaced during the build: the list page size (30) exceeded `noted-rag`’s `top_k <= 20` cap, so every search returned an empty 422 until the backend clamped `top_k`; a `setTimeout` search debounce, ruled out by the project’s no-polling rule, was replaced with explicit Enter-to-search.

4.3 Projects

Conversations are organised into Projects (`chats.js` plus `/api/job2cool/chats/*`), each with a name, optional description, and **private or shared visibility (shared by default)**. Access control is real: private projects live per user; shared projects are listed for everyone but are owner-only for edit and delete. Reopening a project performs full-fidelity replay, restoring the conversation, the workspace documents, the role, and the live Thinking, Graph, and Score panels of every past turn (panel snapshots are persisted with the project and re-rendered through the same widget code path). This is what compensates for the buffer-save stub: work persists in the project even though per-buffer save is deferred.

4.4 Infrastructure and Live Monitoring

job2cool ships as one image (backend plus baked-in frontend) deployed with `docker compose up -d --build job2cool-backend` against the shared stack. A project-wide rule shapes the implementation: **the browser never polls**. `setTimeout` and `setInterval` for data-fetching are banned, and live updates are pushed over SSE for the document buffers and over Socket.IO for KB build progress (`kb:progress`) and infrastructure health.

The Help & Support view is a self-documenting, live-monitored architecture map: a hand-authored, data-driven SVG (no graph library) of sixteen boxes, the browser plus the fifteen request-path containers, deliberately excluding `noted`'s ingestion and training infrastructure (Airflow, Postgres, Redis, MLflow, MinIO, and Evidently), which powers KB operations rather than the request path. Server-side, `health_monitor.py` runs one `asyncio` loop probing about 13 containers every 10s and pushing a `health:status` snapshot to a Socket.IO room, but only while at least one browser is watching (watcher-gated). Each probe maps to a colour: a connection error or timeout is red, an HTTP 5xx is orange, any other response is green. Two cases are special: `websearch_server`, isolated on `mcp_internal`, is observed through a side-effect-free passthrough on `mcp-service`; and the backend stamps itself, so if it dies no push arrives at all, which the client turns into the correct all-red signal.

Detecting the *absence* of an event (staleness) is solved without a timer too. A CSS animation decays a “live feed” badge from green through yellow and orange to red over about 40s, every incoming push restarts the keyframe (an event-driven reset), and if pushes stop the browser's own animation clock carries the badge to red and the `animationend` event dims all dots. The monitoring is liveness, not deep health (a green dot means “responded,” not “every subsystem healthy”), and it is a single-operator dashboard rather than an alerting pipeline, which is appropriate for a demo and recorded as such.

5 Evaluation

5.1 What “the system works” means

For a grounded HR-document generator, success is not a single accuracy number. The evaluation is defined on three axes, each with a measurable signal, plus an operational axis:

1. **Routing correctness:** from a natural-language request, does the system select the right deliverables and the right KB domain? Signals are section-segregation accuracy and domain-resolution correctness, including correct fallback when a family domain is absent from the KB.
2. **Grounding:** are claims backed by verifiable citations? Signals are the resolvable-citation rate (a chunk tag resolving to a valid PDF page) and the RAGAS-style faithfulness and answer-relevance scores from the per-turn judge.

3. **Preservation of the aligned model’s contribution:** once the A2 Job Offer draft passes through `gemma-4`’s composition, is its quality preserved or degraded? This is directly observable through the built-in MA2/Gemma toggle, with the judge scoring both versions independently per turn.
4. **Operational:** per-turn latency and embedder memory under concurrency (Section 3.5).

Pure automatic metrics under-measure a grounded-composition system, and a single LLM judge has a known discrimination ceiling (an A2 finding). The design therefore *triangulates*: a faithfulness judge for grounding, a deterministic citation-resolution check for provenance, observable routing for the agentic behaviour, and a human-readable A/B view for the model-preservation question, with the A1/A2 quantitative baselines anchoring the bottom of the arc.

5.2 Baselines and results

The brief requires isolating the final-stage contribution against (a) the base pretrained model and (b) the A2 aligned model with no system around it. We carry the A1/A2 numbers and define the A3 comparison as (c) the full system.

Table 3: Quantitative results across the three-stage pipeline.

#	Configuration	Metric	Result
(a)	SmolLM2-360M base	In-domain PPL (Djinni)	16.37
(a)	SmolLM2-360M base	Validation PPL	11.66
(b)	<code>ma2-360m-sft</code>	Validation PPL	4.54
(b)	<code>ma2-360m-dpo-b01</code>	Validation PPL	6.01 ($\beta=0.10$)
(b)	<code>ma2-360m-dpo-b01</code> vs SFT	Win-rate (Granite judge, order-swap)	8-0 / 20 prompts
(b)	A1 LoRA merge	In-domain PPL (Djinni)	11.38 (31% below base)
(c)	Full job2cool	Embedder RSS under stress	2,238 MB peak, flat
(c)	Full job2cool	Candidate ingest throughput	100-120 CV/s
(c)	Full job2cool	A2 drafter throughput	430 tok/s, 0.7 s/response

The step from (a) to (b) is real and measured, carried from the earlier reports. The comparison from (b) to (c), whether `gemma-4`’s composition preserves the aligned draft’s quality, is the question this assignment is centrally about, and the system makes it directly observable through the MA2/Gemma toggle and the per-turn judge. **The formal (a)/(b)/(c) baseline run over a fixed prompt set is the remaining evaluation task:** the harness and metrics are defined and the judge is live, but the systematic run and the comparative table are not yet executed. This is stated plainly rather than implied as done, consistent with the honest-reporting requirement.

The following system-level signals are live and verifiable today:

- The RAGAS judge returns faithfulness, answer-relevance, and a rationale per turn.
- Citation resolution is testable end-to-end: graph-grounded chunks resolve to a PDF page and bounding box, inspectable via the Score panel and `/api/graph_trace`.

- Routing is observable from the tabs that appear and from the closing note, which states the resolved domain and flags thin coverage.
- Candidate retrieval is strong on multi-word role phrases (“senior python backend engineer with fastapi” returns top hits of 0.99, 0.97, and 0.96, all genuine Python backend engineers) and honestly weak on bare tokens or out-of-era vocabulary (“genai” returns about 0.000x, because the 2020–2023 corpus predates LLM-engineering terminology, a retrieval-quality finding rather than a bug).
- Performance is measured: 100–120 CV/s bulk-embed throughput, the embedder OOM fixed and stress-verified flat at about 2.2GB, and the full 210,048-CV ingest completed with read-back acknowledgement equal to the indexed count.

5.3 Qualitative analysis

The report will include concrete turns illustrating each category:

- **Success.** “Backend developer on AWS”: the role is extracted, `jobs_onboard_backend` resolves, concurrent hybrid RAG runs, and four sections are generated with resolvable bbox citations. Routing, grounding, and provenance end to end.
- **Fallback.** An embedded-systems role whose `jobs_onboard_embedded` domain is absent falls back to `general`; the closing note surfaces the gap explicitly, honest gap-reporting as a correct behaviour.
- **Memory.** “I need a DevOps engineer” followed by “now make the onboarding remote-friendly”: `_resolve_need` folds the prior role in, and the follow-up updates only the Onboarding Plan.
- **Refusal.** “I need to hire someone” carries no concrete role, so Diana asks which role and generates nothing.
- **MA2 preservation.** The same offer via the MA2 toggle versus the gemma-composed version, judged side by side, is the (b)-versus-(c) evidence for Section 7. Qualitatively the A2 draft is structurally clean and domain-specific; the composed version keeps that structure while integrating retrieved evidence and citations.

6 Critical Discussion

Honest reporting of failures and limitations is explicitly rewarded by the brief, so this section is deliberately candid.

6.1 What worked

The end-to-end pipeline is verified: a plain-language request flows through conversational memory, role, section, and domain resolution, the A2 draft plus **gemma-4** composition and hybrid RAG, and into live cited tabs. Citations resolve to a PDF page and bounding box for graph-grounded chunks. Agentic routing behaves as designed, with section classification, the ask-back guard, and the fallback domain chain all holding. The 210,048-CV corpus is vector-indexed and read-back verified, with a working Candidates browser over it. Shared services were extracted cleanly (**kb-service**, **mcp-service**, and the isolated **websearch_server**); Projects provide private and shared access with full-fidelity replay; and the infrastructure health map is live, event-driven, and self-documenting. The shared-embedder OOM was root-caused, fixed with a one-line change, and stress-verified flat under the exact concurrency that had crashed it four times, a real engineering outcome rather than a workaround.

6.2 Honest limitations

The **single largest gap** is that the **match_candidates** tool and the conditional composition (Section 3.6) are not wired. What exists is the retrieval substrate and a human-facing browser; the threshold-gated branch and the seventh “Candidate Matches” bundle section are deferred. Other limitations, in rough order of impact, are:

- **Per-section KB domains are not differentiated.** All four sections ground on the single resolved onboarding domain; dedicated interview-bank, culture, and benefits corpora are not yet provided, so for example Cultural and Team Fit grounds on onboarding material, an approximation rather than the final design.
- **One onboarding family is missing** (embedded); ten of eleven exist.
- **The formal baseline run is outstanding.** The harness, metrics, and judge are in place, but the systematic (a)/(b)/(c) table is not yet executed (Section 5).
- **gemma-4 judges its own output**, with no cross-judge isolation, doubly relevant given the A2 judge-discrimination ceiling.
- **Two wrong diagnoses preceded the OOM fix** (token-budget and Cyrillic theories), each announced as a fix that did not hold, reported rather than hidden.
- **Buffer-save is a stub**; work survives via Projects replay, but per-buffer persistence is incomplete.
- **Audio is proxy-only**: STT, TTS, and avatar route through the deployed origin, not `localhost:4920`.
- **Company Profile** is a placeholder, and a full browser visual-verification pass (split-PDF render, voice on the proxy, project replay) is the standing manual QA item.

6.3 Trade-offs

think=False on utility calls is necessary, since **gemma-4** otherwise returns empty after spending its budget on reasoning, but it denies those classification and summarisation steps any chain-of-thought, possibly capping their quality: a genuine capability-versus-stability trade-off.

LLM-decided pipeline versus predictability. The design is flexible for ambiguous and iterative requests but harder to test deterministically and occasionally mis-routes. The keyword-heuristic failure that motivated the LLM classifier shows the opposite failure mode: the LLM is more reliable, but its mistakes are less predictable.

Shared-service reuse versus autonomy. The copy-only strategy saved enormous build effort but created an infrastructure dependency, forced additive-only changes (new endpoints rather than edits), required one authorised exception, and placed the bulk ingest on a shared embedder whose concurrency behaviour caused the OOM.

Slot cap: memory versus parallelism. Capping bge-m3 to one slot bounds RSS under any load but serialises concurrent embeds. For this workload of modest concurrent traffic it is the right call; on a write-heavy multi-tenant embedder it might not be.

Two co-resident models versus one. Keeping the 360M specialist alongside **gemma-4** costs a little GPU residency but preserves the arc: the A2 model stays a live component rather than dissolving into a larger model.

6.4 What we would do differently, and next steps

The priority order is to wire **match_candidates** and the conditional composition (the largest value step, with the substrate already in place); to differentiate KB domains per section and add the missing **embedded** domain; to run the formal (a)/(b)/(c) evaluation and publish the table; to move the judge to a separate slot to restore cross-judge isolation; to persist buffers and finish the browser-verification pass; to activate the Company Profile surface; and to consider a unified post-merge rerank across vector and graph evidence.

7 Reflection on the Three-Stage Arc

This reflection is part of the grade: which stage contributed most, and were earlier stages washed out by later ones?

The A1/A2 work is directly visible, not diluted. **ma2-360m-dpo-b01** is literally the Job-Offer drafter inside the running system, not a discarded checkpoint. A1 made a 360M model fluent in job-posting language (in-domain perplexity 16.37 down to 11.38); A2 taught it to follow recruiter instructions and aligned its output to a quality rubric (SFT validation perplexity 4.54; DPO-b01 the $\beta=0.10$ deliverable, 8 wins to 0 against SFT); and A3 placed that specialist inside a system that can finally use it on a real task.

Did RAG make domain adaptation redundant? No: they operate on different axes. A1/A2 improved the writing quality and structure of one output in isolation, while RAG supplies what no amount of fine-tuning a 360M model can give, namely knowledge of *this company's* actual material and verifiable provenance (a claim traced to a PDF page and bbox). A fine-tuned model can write a fluent onboarding plan; it cannot write this company's onboarding plan, nor prove where a requirement came from. They are complementary, not substitutes.

Was the A2 alignment washed out by gemma's composition? This is the sharpest question, and the system is built to answer it: when both are selected, the raw A2 draft and the gemma-composed Job Offer are produced side by side (the Section 5 (b)-versus-(c) view). The honest expectation, given a 4B composer refining a 360M draft, is *some* masking, with the final prose leaning toward gemma-4. But the A2 model's contribution is structural: it reliably produces the four required sections in the required Markdown, aligned to recruiter preferences, and that structure persists as the seed the composer refines. The value of A2 in this architecture is a fast (about 0.7s), structurally-correct, aligned seed that anchors the composition, not the source of the final wording. Reporting that masking openly, rather than claiming the 360M model dominates the output, is the honest reading.

A1's marginal contribution under a 4B model with RAG. The most genuinely open question is whether A1's raw posting-fluency contribution stays clearly non-redundant once gemma-4 and RAG are in the loop, since retrieved company postings could in principle teach the composer the style A1 baked into the 360M model. A1's contribution is most clearly non-redundant for the A2 instruction-following and alignment it enabled, and least clearly so for raw fluency in this configuration, a tension worth stating rather than papering over, and one a future A1/no-A1 toggle could quantify just as the MA2/Gemma toggle quantifies the A2 question.

Which stage contributed most? For the final system's *usefulness*, A3 contributed most: it is the orchestration, retrieval, grounding, and platform that turned an aligned drafter into something a recruiter can operate. For the specific quality of the Job Offer deliverable, A1 and A2 contributed most. These are not in competition. An aligned model with no system is a good drafter no one can operate; a system with no aligned model has no specialist at its core. The most defensible single sentence is that the final stage did not make the earlier stages redundant: it gave them somewhere to be useful.

8 Conclusion

job2cool is a functional system, runnable end-to-end with two clear entry points (a recruiter-facing web UI and a CLI ingestion tool), that turns a recruiter's natural-language request into a grounded, citable pack of HR documents built on `ma2-360m-dpo-b01`, the model produced

across Mini-Assignments 1 and 2. On top of the fine-tuning and alignment it integrates four additional course components, namely hybrid RAG with PDF and bbox citations, a multi-step LLM-decided pipeline, a per-turn LLM-as-judge, and quantised inference with a root-caused performance fix, plus chain-of-thought as a supporting fifth. The whole runs across fifteen open-source, request-path containers with Google identity, live health monitoring, a 210,048-entry candidate CV corpus, and full-fidelity project replay; no closed API appears anywhere in the serving path.

The arc is coherent and honest: the aligned A2 model is not replaced but kept as a live, visible specialist, and the final stage's value is in making it usable, grounded, and measurable rather than in superseding it. The main limitations, namely the unwired `match_candidates` integration, the outstanding formal baseline run, incomplete per-section domain differentiation, and self-judging evaluation, are reported plainly. The most important next step is wiring candidate matching into the agentic composition pipeline, where the retrieval infrastructure is already complete and verified.